



Digitális Kultúra Sprint

Utolsó simítások

I. Stratégia és Időkezelés

A Sikeres Vizsga Alapkövei

Hogyan kezd el?

- ✓ **Olvasd el:** Az első 10 percben csak olvasd át az összes feladatot!
- ✓ **Forrásfájlok:** Ellenőrizd, hogy minden megvan-e a mappában.
- ✓ **Szerkezet:** Hozz létre tiszta mappastruktúrát a mentéshez.

Kritikus hibák elkerülése

- ✓ **Mentés:** 15 percenként Ctrl+S. A fájlneveket ne módosítsd feleslegesen!
- ✓ **Relatív utak:** HTML/Python esetén soha ne használj abszolút útvonalat.
- ✓ **Ellenőrzés:** A vizsga vége előtt 20 perccel hagyd abba a kódolást és ellenőrizz!

Kapkodásból adódó bakik

Mire figyelj?

- ✓ Olvasd át, húzd alá a lényegét!
- ✓ Ellenőrizzük le, hogy tényleg azt kérte-e a feladat, fontos a LEGALÁBB/LEGFELJEBB! (egyenlőséget megengedünk ilyenkor)

Időbeosztás

- ✓ Kezdjük a számunkra legkönnyebb feladattal!
- ✓ 60 perc után menjünk a következő témakörre, ne időzzünk el sokat egynél, majd visszatérsz!

II. Táblázatkezelés (Excel)

Legfontosabb Függvények

Kulcsfüggvények

- ✓ **XKERES**
- ✓ **HA() / ÉS() / VAGY()**: Összetett logikai feltételek kezelése.
- ✓ **DARABHA / SZUMHA**: Feltételes statisztikák alapjai.
- ✓ **KÖZÉP()** függvény



Aranyszabály: Mindig gondold át a hivatkozást!
Abszolút (\$A\$1), Relatív (A1) vagy Vegyes (\$A1).

III. Adatbázis-kezelés (SQL)

SQL Záradékok és Műveletek

Záradékok & Operátorok

- **SELECT / DISTINCT:** Mezők / egyedi értékek
- **FROM / WHERE:** Tábla / feltétel
- **GROUP BY / HAVING:** Csoportosítás / aggregált feltétel
- **ORDER BY:** Rendezés (DESC: csökkenő)
- **LIMIT / OFFSET:** Korlátozás / kihagyás

Logikai: AND, OR, NOT

Összehasonlító: =, <>, >, <

Függvények & Szűrés

- **Aggregáció:** SUM, AVG, MIN, MAX, COUNT(*)
- **Dátum:** YEAR, MONTH, DAY, NOW()
- **Szöveg:** CONCAT, LOCATE
- **Logikai:** IF(felt; igaz; hamis)

Predikátumok & Jokerek

IN(...), BETWEEN x AND y, IS NULL

LIKE '_a%' (_: 1 jel, %: bármilyen)

Adatmódosítás (DML)

INSERT: Rekord beszúrása

```
INSERT INTO `t` (`m1`) VALUES (e1);
```

UPDATE: Módosítás

```
UPDATE `t` SET `m1` = uj WHERE id=9;
```

DELETE: Törlés

```
DELETE FROM `t` WHERE id=9;
```

⚠ FIGYELEM: UPDATE és DELETE esetén soha ne felejtse el a WHERE-t!

OR hatóköre

Ha a feladat szövegében azt látod, hogy van **két alternatíva (A vagy B csoport)**, és az egész csoportra igaznak kell lennie **még egy plusz feltételnek (C)**, akkor a „VAGY”-os részt **MINDIG** tedd zárójelbe:

WHERE (A OR B) AND C

Feladat: Listázzuk ki azokat a termékeket, amelyek **Tej** vagy **Sajt** megnevezésűek, **ÉS** az áruk 500 Ft felett van.

✗ A hibás megoldás (Zárójel nélkül):

SQL



```
WHERE nev = 'Tej' OR nev = 'Sajt' AND ar > 500
```

- **Mit csinál ezzel a gép?** Mivel az **AND** az erősebb, először összeköti a Sajtot az árral.
- **A gép ezt olvassa:** „Add ide azokat a Sajtokat, amik 500 Ft felett vannak... **VAGY** add ide az **ÖSSZES Tejet**, teljesen függetlenül attól, hogy mennyibe kerül!” (Tehát a 100 Ft-os tej is megjelenik a listában).

OR hatóköre folytatás

A helyes megoldás (Zárójellel):

SQL



```
WHERE (nev = 'Tej' OR nev = 'Sajt') AND ar > 500
```

- **A gép ezt olvassa:** „Először megnézem a zárójelet: a termék legyen Tej vagy Sajt. Ha ez stimmel, akkor a következő lépésben kikötöm, hogy ennek a csoportnak az ára kötelezően 500 Ft felett legyen.”

Tehát mindig zárójelezzük a részt amire vonatkozik a VAGY!

Belseő select/Allekérdezés HIBÁK

```
-- HIBÁS MEGOLDÁS!  
WHERE vevo_id = (SELECT id FROM vevok WHERE varos = 'Debrecen')
```

Miért hibás? Ha Debrecenben több vevő is lakik (ami elég valószínű), az alkérdezés egy listát ad vissza (pl. 4, 12, 28). A gép nem tudja értelmezni azt, hogy `vevo_id = [4, 12, 28...]`. Az egyenlőségjel csak **egyetlen egy** dologra tud vonatkozni. Viszont amint meglátja az IN-t, rögtön tudja, hogy a listában keresgéljen.

```
SELECT rendeles_id, datum  
FROM rendelesek  
WHERE vevo_id IN (SELECT id FROM vevok WHERE varos = 'Debrecen');
```

A zárójelek: Az alkérdezést **MINDIG** teljesen zárójelbe kell tenni, különben az SQL nem különíti el a főlekérdezéstől.

IV. Algoritmizálás (Python)

Fájlbeolvasás

Szótár használata opcionális,
viszont ajánlott, így elkerülheted,
hogy később összekevered, melyik
index mit jelent

```
file = open("bedat.txt", "r", encoding="utf-8").read().splitlines()

tanulok = []
for sor in file:
    sor = sor.split()
    tanulo = {}
    tanulo["kod"] = sor[0]
    tanulo["ido"] = sor[1]
    tanulo["esemeny"] = int(sor[2])
    tanulok.append(tanulo)
print(tanulok)
```

Vagy hasonlóan a with open() használatával

Python Típusalgoritmusok



Összegzés

Elemek összeadása vagy
átlagolása.



Keresés

Van-e ilyen elem, és ha igen,
hol? (break használata a
ciklusban HA CSAK AZ ELSŐRE
kíváncsiak).



Kiválogatás

Feltételnek megfelelő elemek
kigyűjtése egy új listába.

Használj f-stringet a kiíratáshoz: `print(f"Az eredmény: {változo}")`

V. Az Utolsó 15 Perc

- ✓ **Mentés:** Minden fájl a megfelelő helyen van?
- ✓ **Pontosan azt a fájlnévet használd amit a feladat kér!** Pl. beleptetes.py
- ✓ **Töröld ki a felesleges "teszt" print()-eket,** amik csak neked kellettek fejlesztés közben.
- ✓ **Excel:** ha képletet másoltál, nem csúszott el a hivatkozás? Jól használtad a \$ jeleket?
- ✓ **SQL:** Minden megoldott feladat szerepel a megadott néven? Pl. 2varos.sql? Figyelj az sql kiterjesztésre!



Ha valami nem fut le:

- 1.) mély levegő, ne essünk pánikba! 4-4-4 légzőgyakorlat!
- 2.) nézd meg nem-e egy elgépelt változó, vagy hiányzó kettőspont, vagy elfelejtettünk int()-elni egy számnak tűnő szöveget!!
- 3.) SQL-ben és pythonban is nagyon sokat segít a piros hibaüzenet, higgadtan olvassuk végig és értelmezzük!
- 4.) Bontsd le atomjaira: **csak az a) vagy az 1. pontot olvasd el.** Csak azt csináld meg. Aztán a következőt. A részpontok is pontok!

Gyakori hibák

-Python-

- Behúzási hiba

- `for tanulo in tanulok:`

- `→tanulok.append(tanulo)`

- Nézd végig függőlegesen a kód szélét. Ami a ciklushoz tartozik, az legyen egy oszloppal beljebb! Ugyanígy az if-nél is!

Gyakori hibák

-Python-

- `INT(INPUT())`
- Amikor bekérünk egy számnak tűnő adatot a felhasználótól ne felejtük el int-té alakítani!!!

Gyakori hibák

-Python-

- Index vs. sorszám!!
- Ne felejtsük el, hogy a pythonban az indexelés 0-tól kezdődik!
- $\text{Sorszám} = \text{index} + 1$
- $\text{Index} = \text{sorszám} - 1$

```
sorszam = int(input("Adja meg a szék sorszámát (1-50): "))
index = sorszam - 1 # Itt történik a varázslat

# Innentől már csak az indexet használod:
print(f"A keresett adat: {adatok[index]}")
```

Kérdések

Olvass figyelmesen el mindent, maradj nyugodt,
és bízz a logikádban!

Sok sikert a vizsgához!

Digitális Kultúra 2026 tavasz
